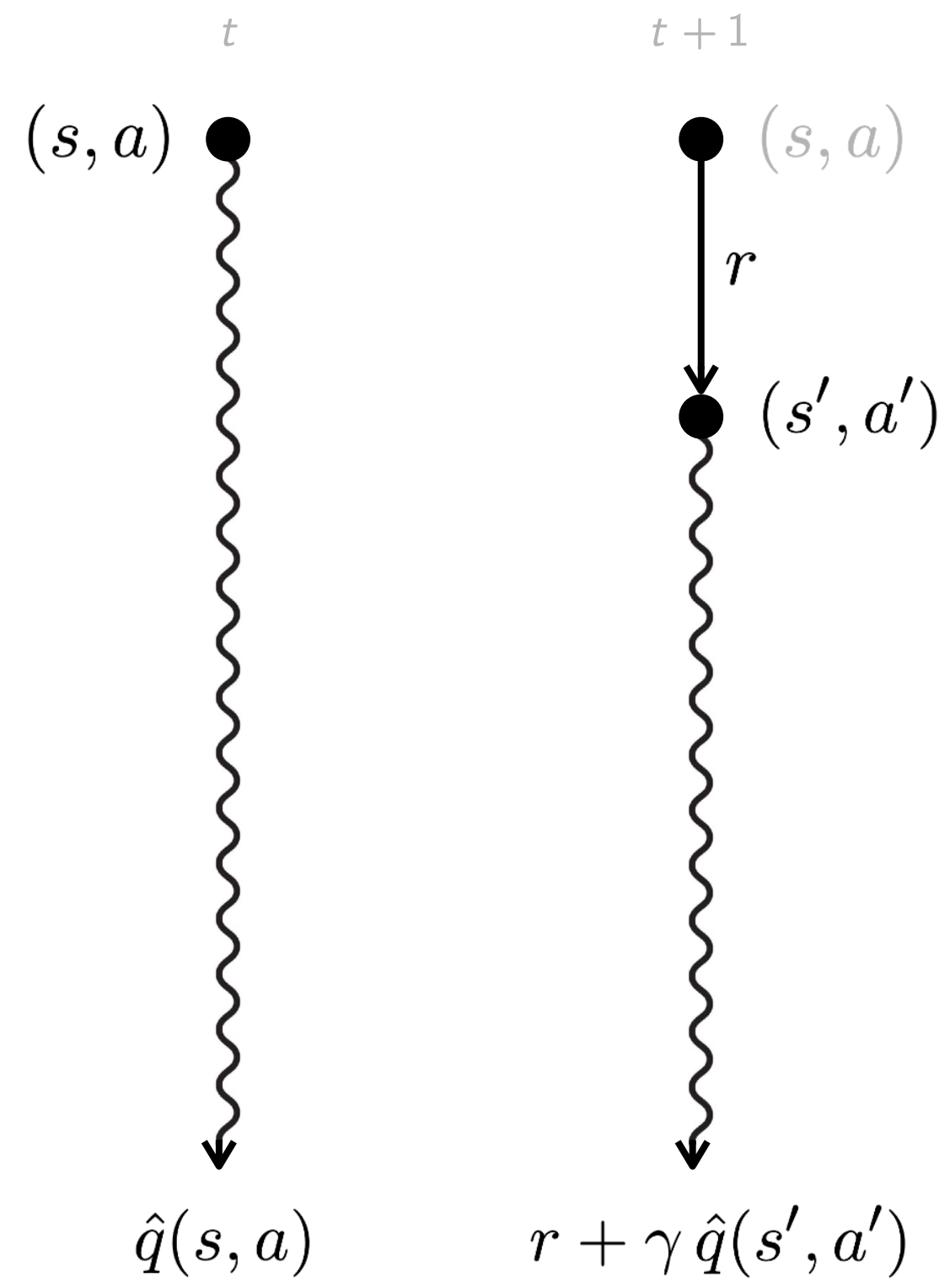


Reinforcement Learning

M.Sc. Artificial Intelligence

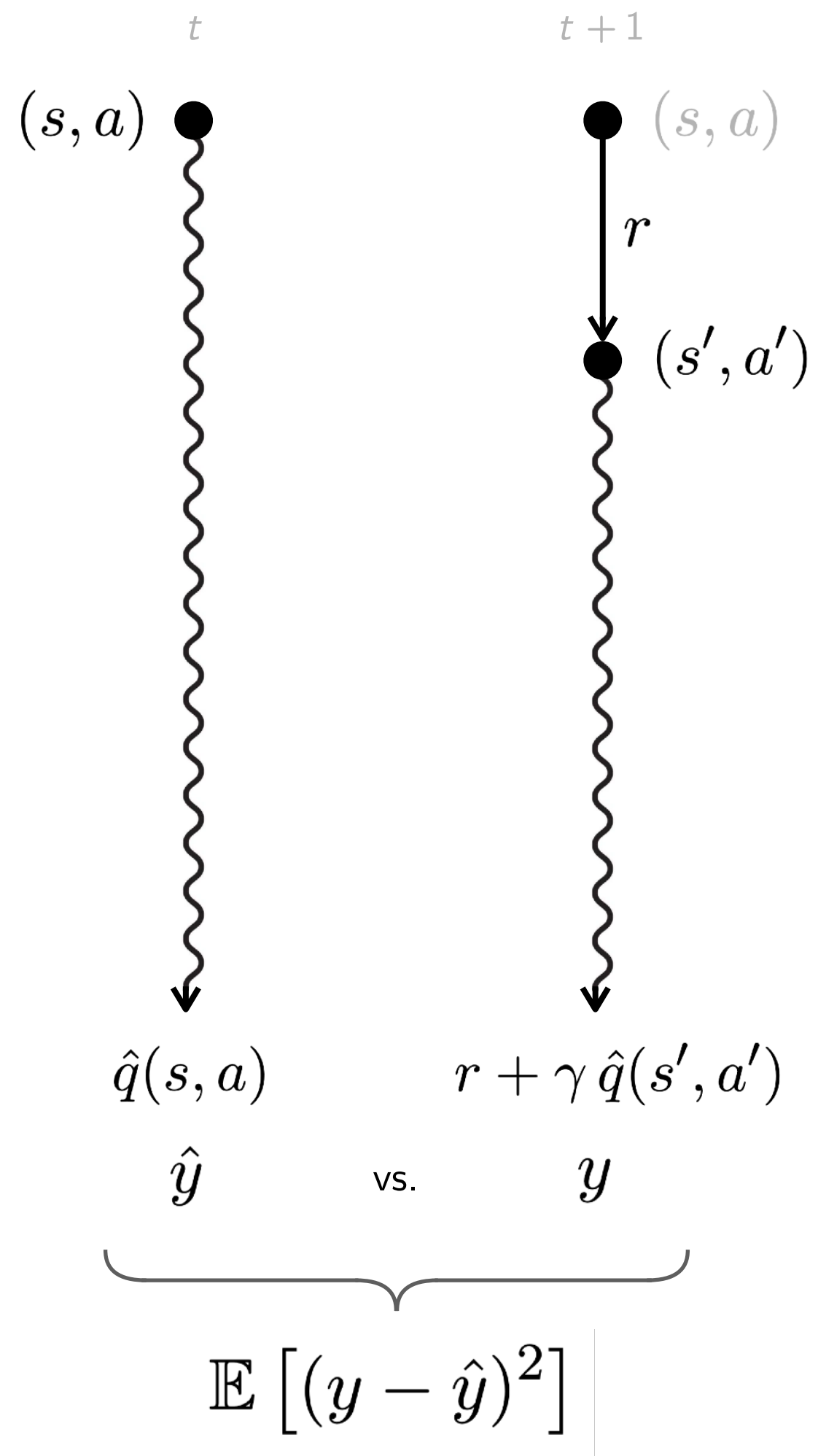
Janik Euskirchen

1. Translate a simple real-world decision problem into an MDP format.
2. (Hand-)Compute the state values of a simple (+- 6 state) MDP for a given fixed policy.
3. Prove/Disprove optimality of a given policy in a simple MDP.
4. Use a dynamic programming technique to find the optimal state-values for a given MDP.
5. Use optimal state-values to generate an optimal policy.
6. Implement a table-based Q-learning algorithm.
7. Implement a table-based on-policy Temporal Difference algorithm.
8. Implement at least two exploration strategies.
9. Show the influence of the degree of exploration on the learned policy when using an on-policy RL algorithm.
10. Illustrate the difference between learned policies when using on-policy and off-policy algorithms.
11. Implement a TD(I) algorithm using eligibility traces.
12. Add (simple/linear) generalization to a Temporal Difference algorithms.
13. Implement a policy gradient algorithm for a contextual continuous action task (inverted pendulum or similar).
14. Successfully train a deep reinforcement learning method
15. Evaluate and compare the performance of a deep reinforcement learning method on at least one environment.
16. Explain the inner workings of a deep reinforcement learning algorithm and its specific interactions with an environment.



Q-Learning:

$$a' = \arg \max_{a'} \hat{q}(s', a')$$



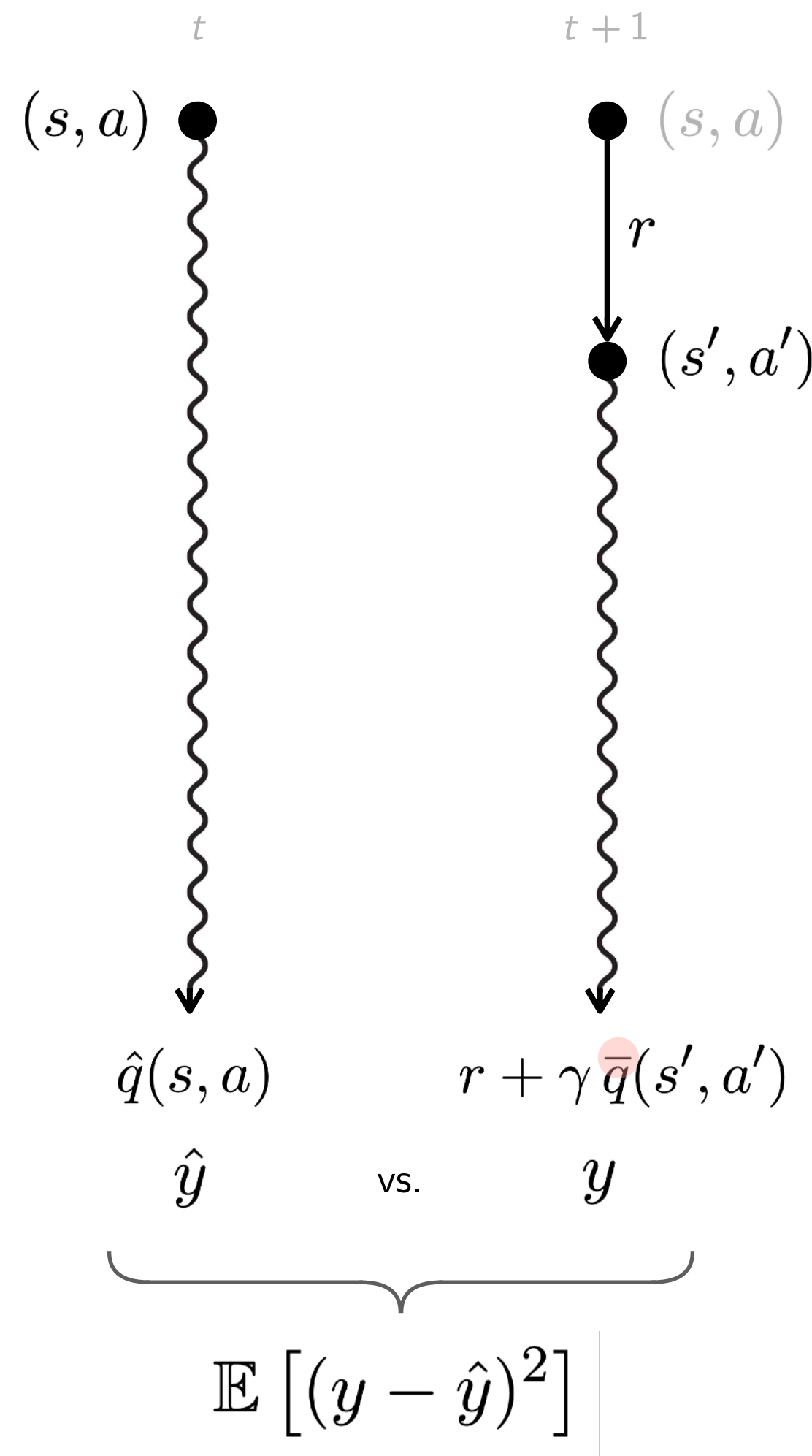
Q-Learning:

$$a' = \arg \max_{a'} \hat{q}(s', a')$$

Optimize parameters:

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \mathbb{E} [(y - \hat{y})^2]$$

$$\nabla_{\mathbf{w}} \mathbb{E} [(y - \hat{y})^2] = \mathbb{E} [(y - \hat{y}) \nabla_{\mathbf{w}} \hat{y}]$$



Q-Learning:

$$a' = \arg \max_{a'} \bar{q}(s', a')$$

Target and prediction networks:

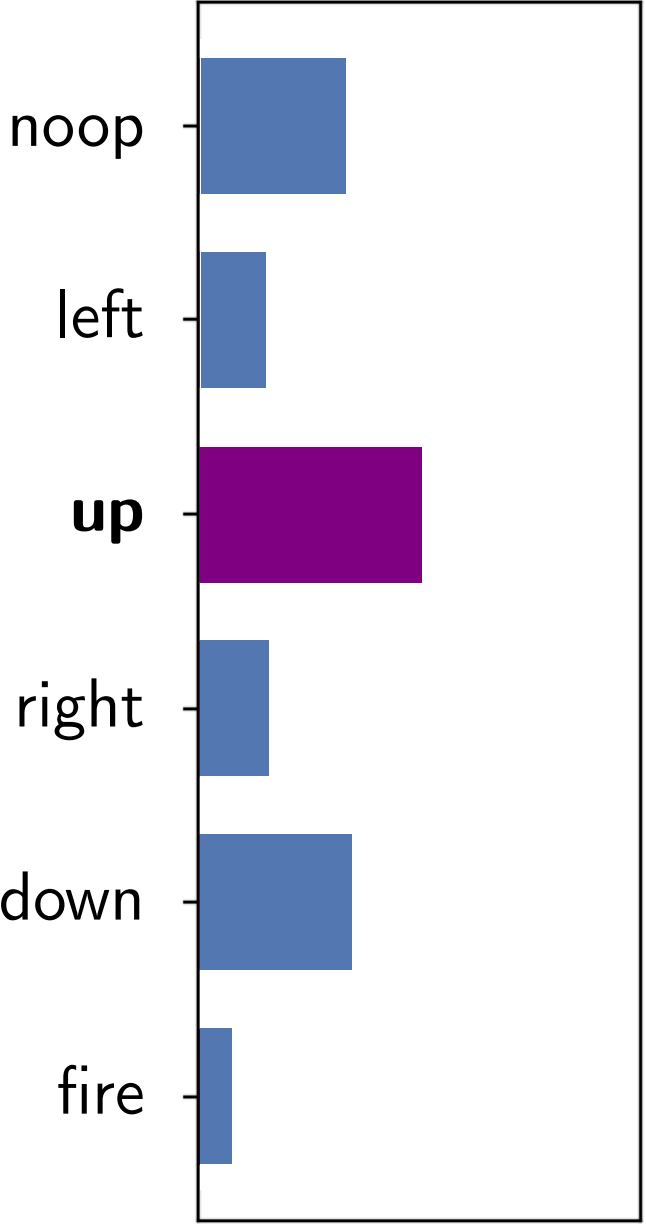
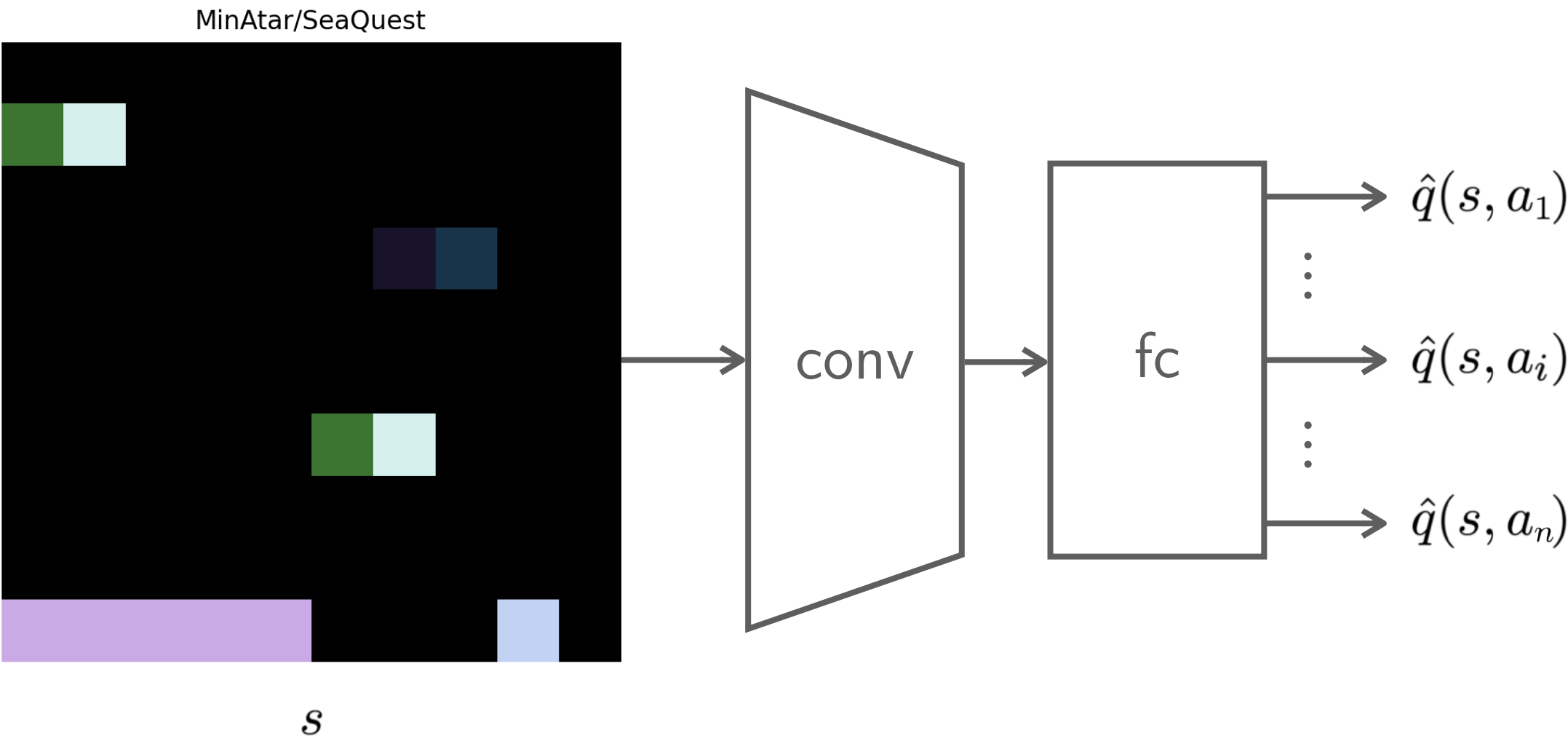
$$r + \gamma \bar{q}(s', a'; \bar{\mathbf{w}})$$

$$\hat{q}(s, a; \mathbf{w})$$

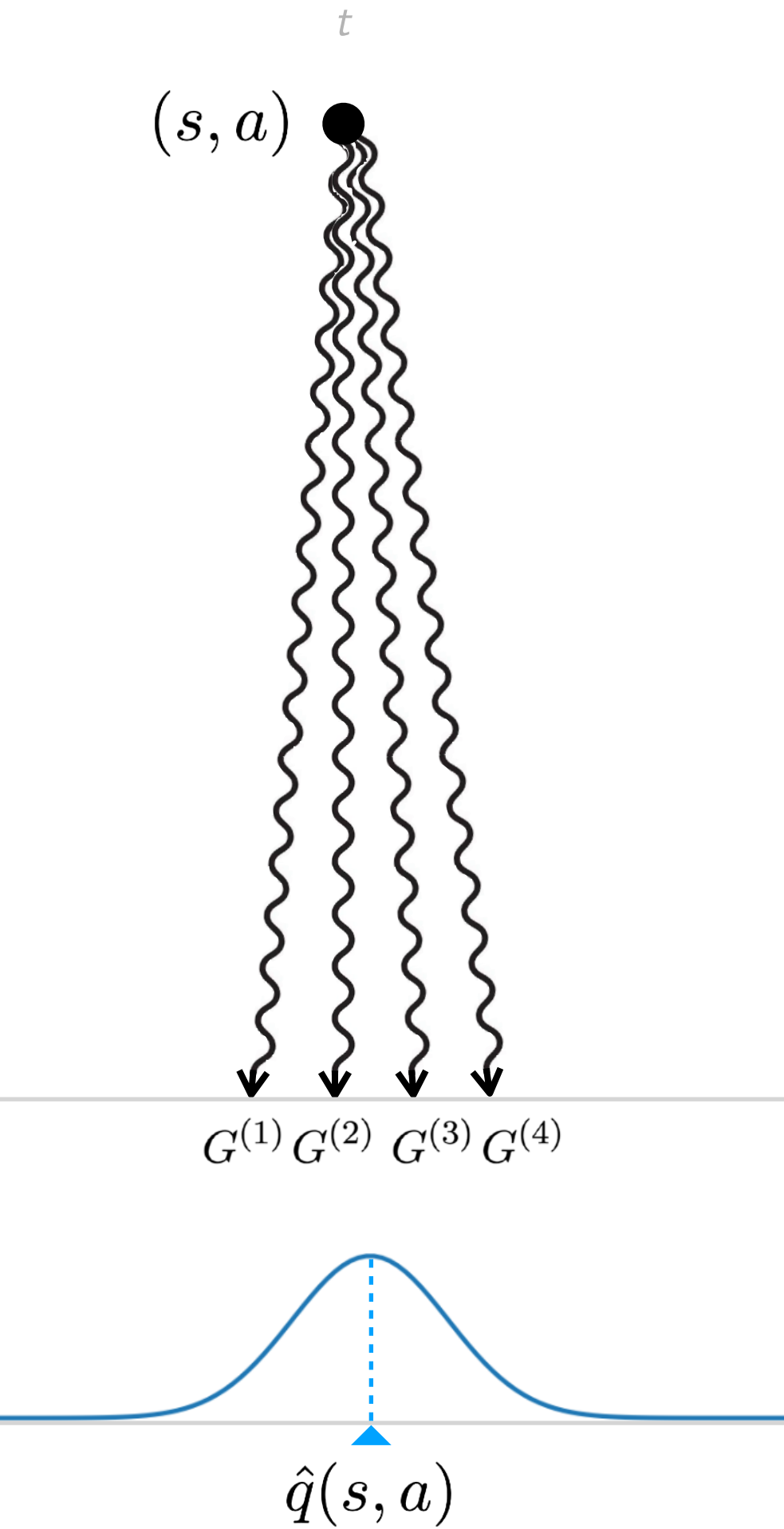
Optimize parameters:

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \mathbb{E} [(y - \hat{y})^2]$$

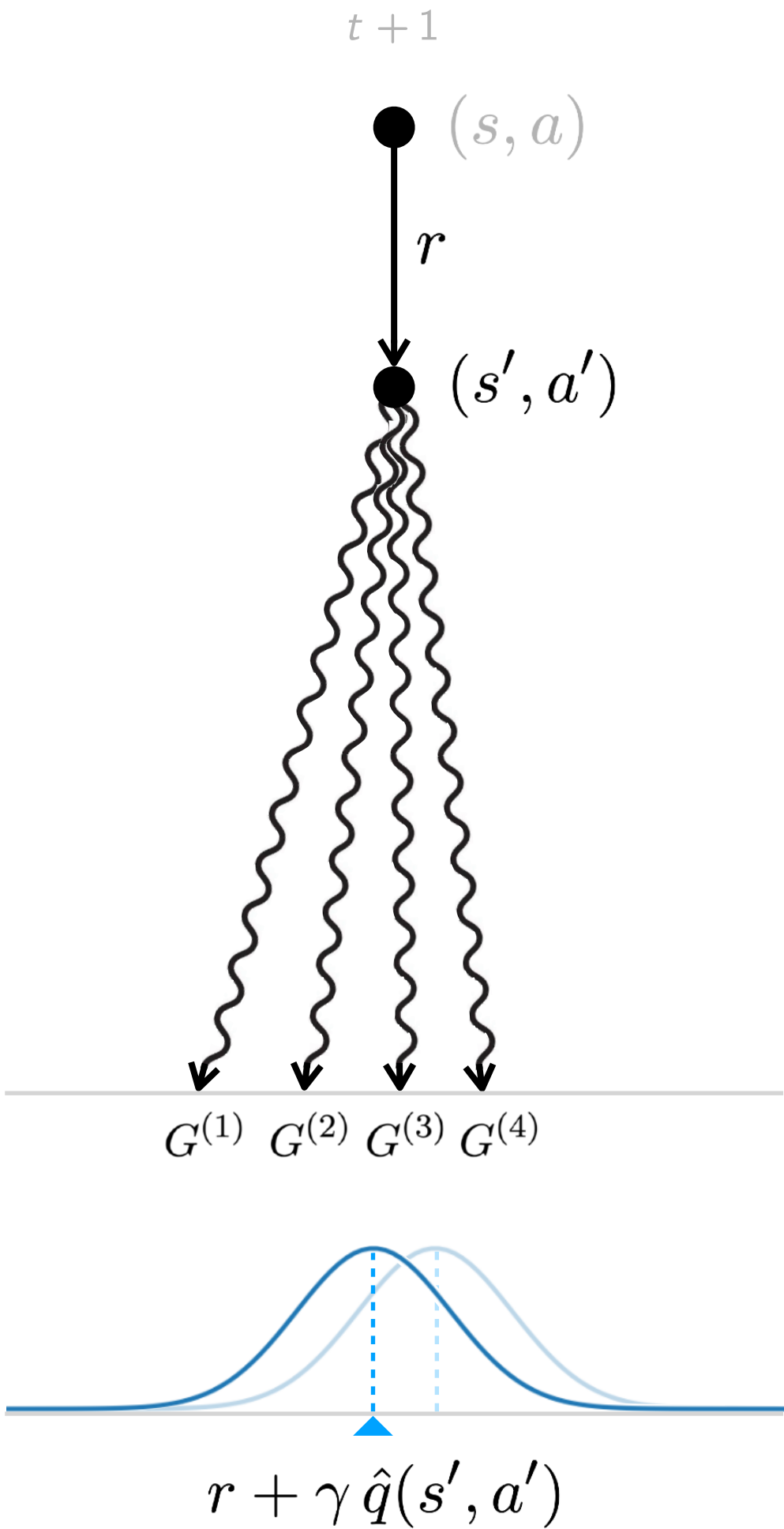
$$\nabla_{\mathbf{w}} \mathbb{E} [(y - \hat{y})^2] = \mathbb{E} [(y - \hat{y}) \nabla_{\mathbf{w}} \hat{y}]$$



Multiple rollouts just to show that the returns can end up being one of many values, and they follow some distribution.

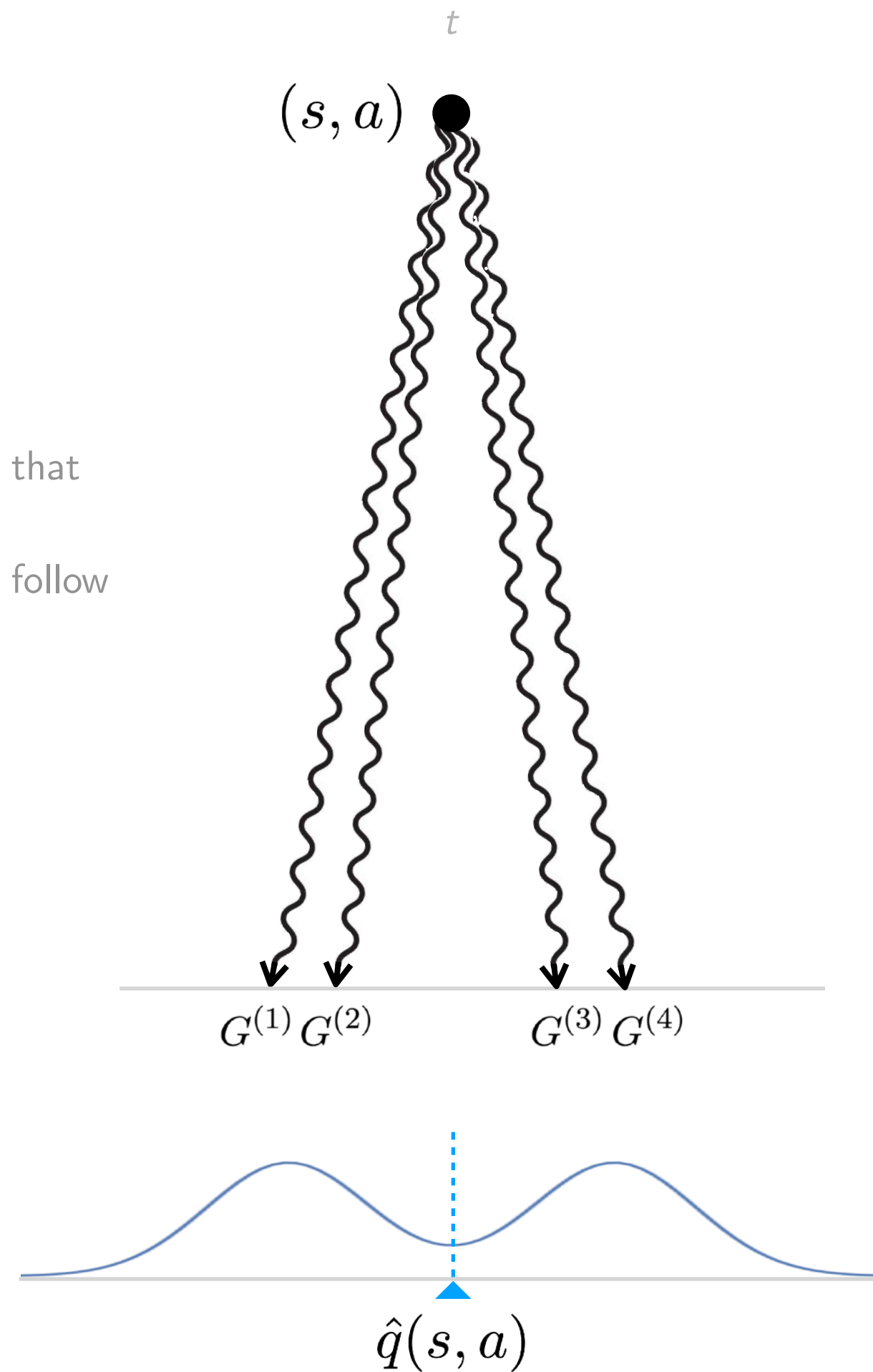


$$q(s, a) = \mathbb{E}[Z(s, a)]$$



$$q(s, a) = \mathbb{E}[r + \gamma \hat{Z}(s', a')]$$

Multiple rollouts just to show that the returns can end up being one of many values, and they follow some distribution.



We still end up taking the expected value of each action, and then selecting the action that maximizes the expected return.

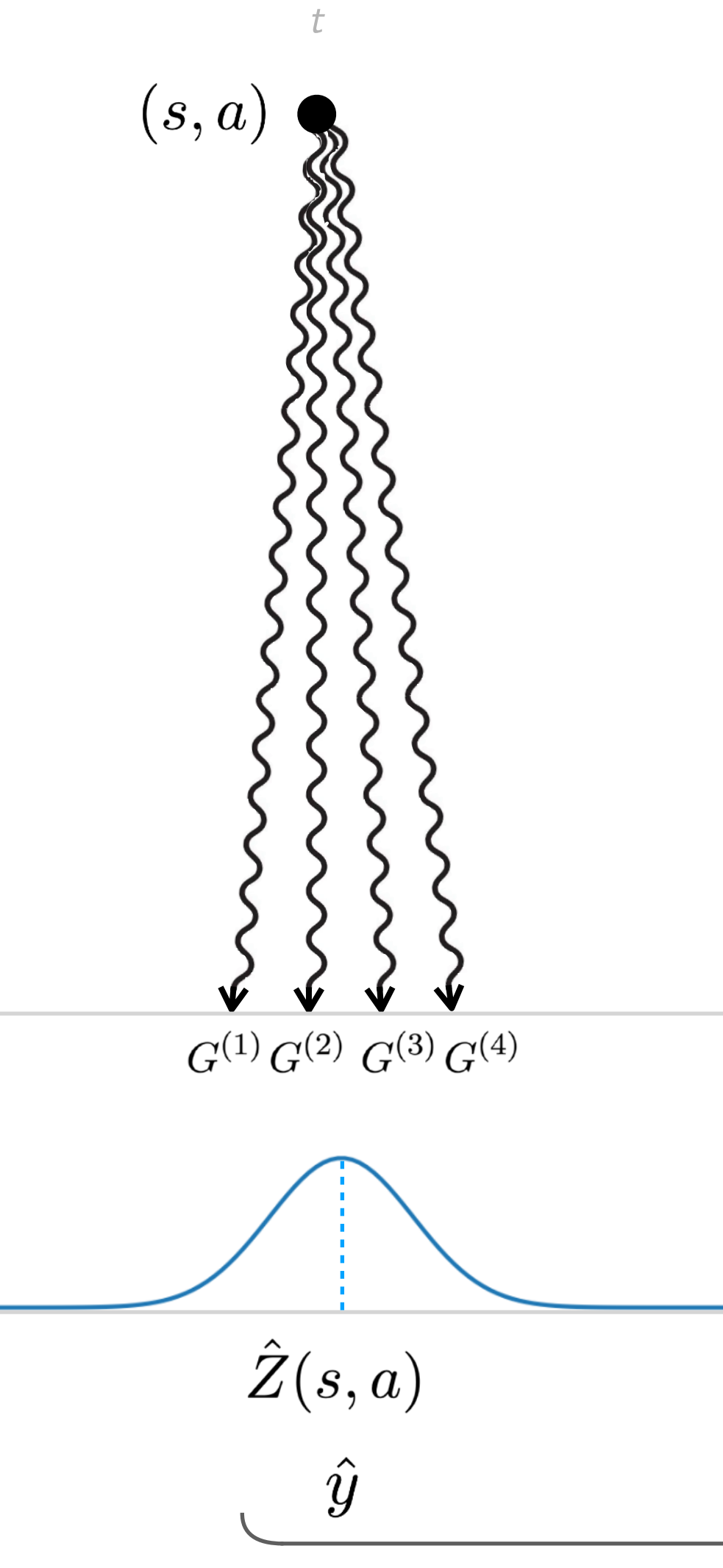
But having the network learn the entire value distribution, and using the cross entropy loss between the predicted distribution, $Z(s, a)$, and target distribution, $r + \gamma Z(s', a')$, provides a richer learning signal! We are forcing the network to learn much more explicitly about which returns will result from which state-action pairs.

At inference time, we can also get a better idea of how uncertain the network is about its value predictions.

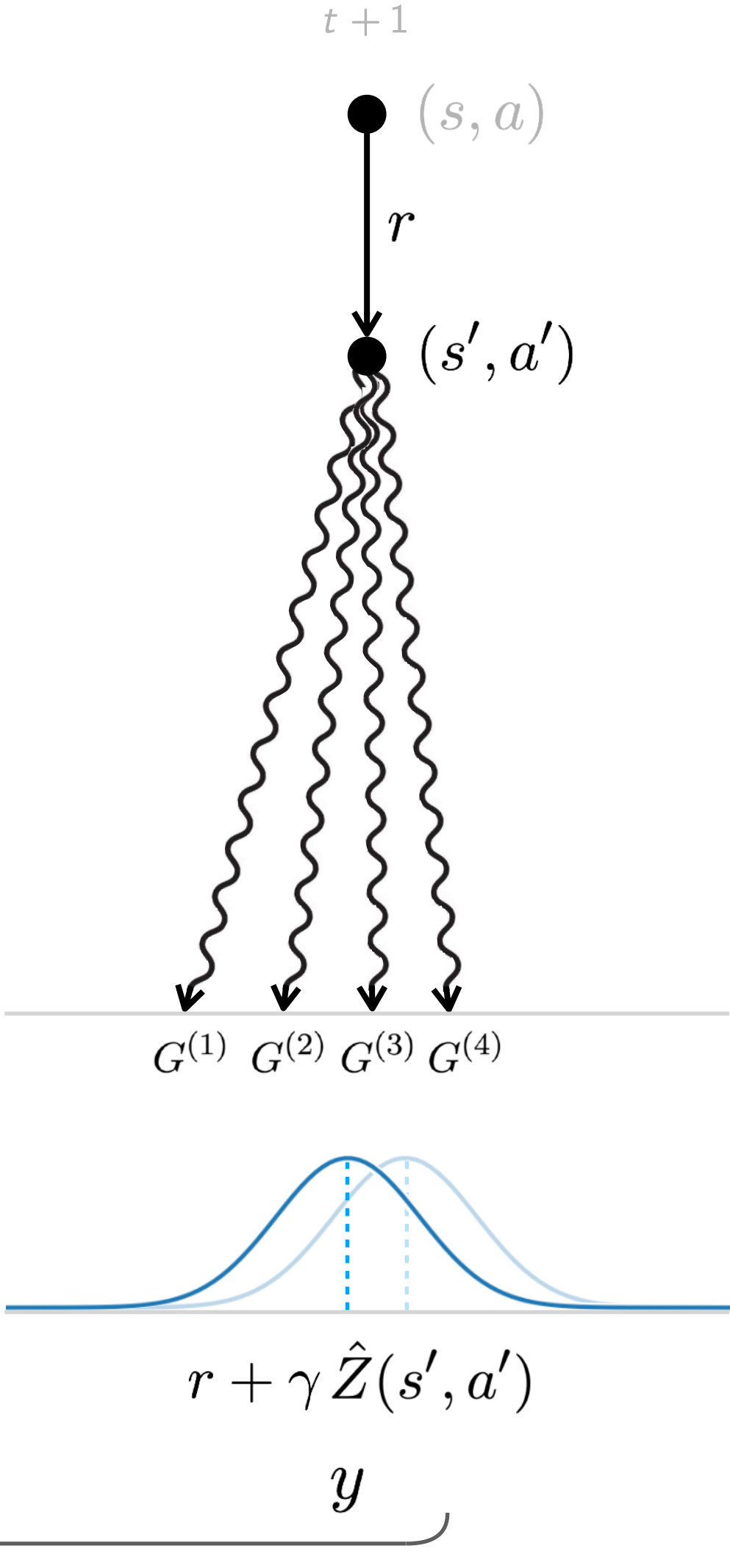
Distributional Bellman equation

$$Z(s, a) \stackrel{D}{=} R(s, a) + \gamma Z(S', A')$$

Multiple rollouts just to show that the returns can end up being one of many values, and they follow some distribution.



Value distribution:

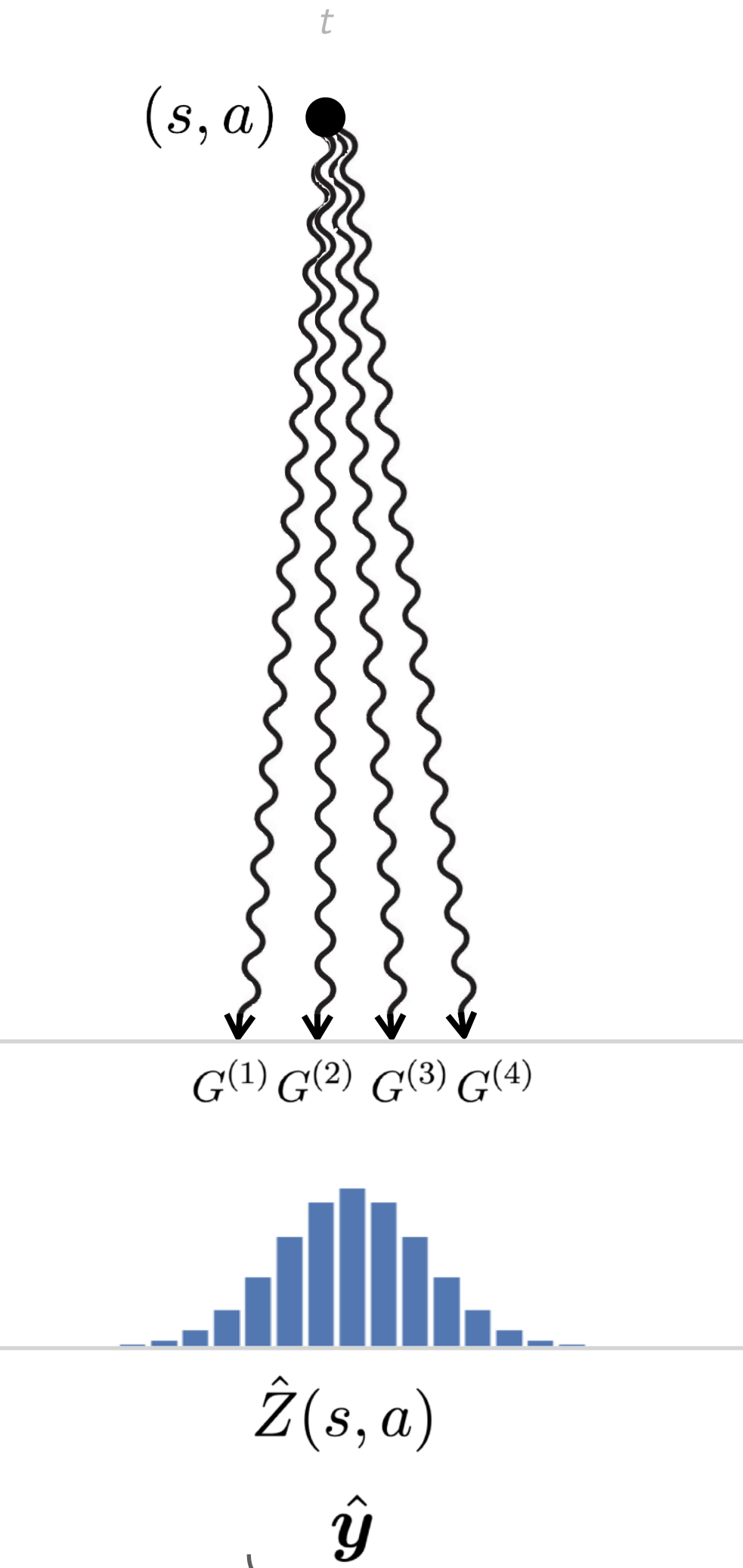


$$\begin{aligned} a' &= \arg \max_{a'} \bar{q}(s', a') \\ &= \arg \max_{a'} \mathbb{E} [\bar{Z}(s', a')] \end{aligned}$$

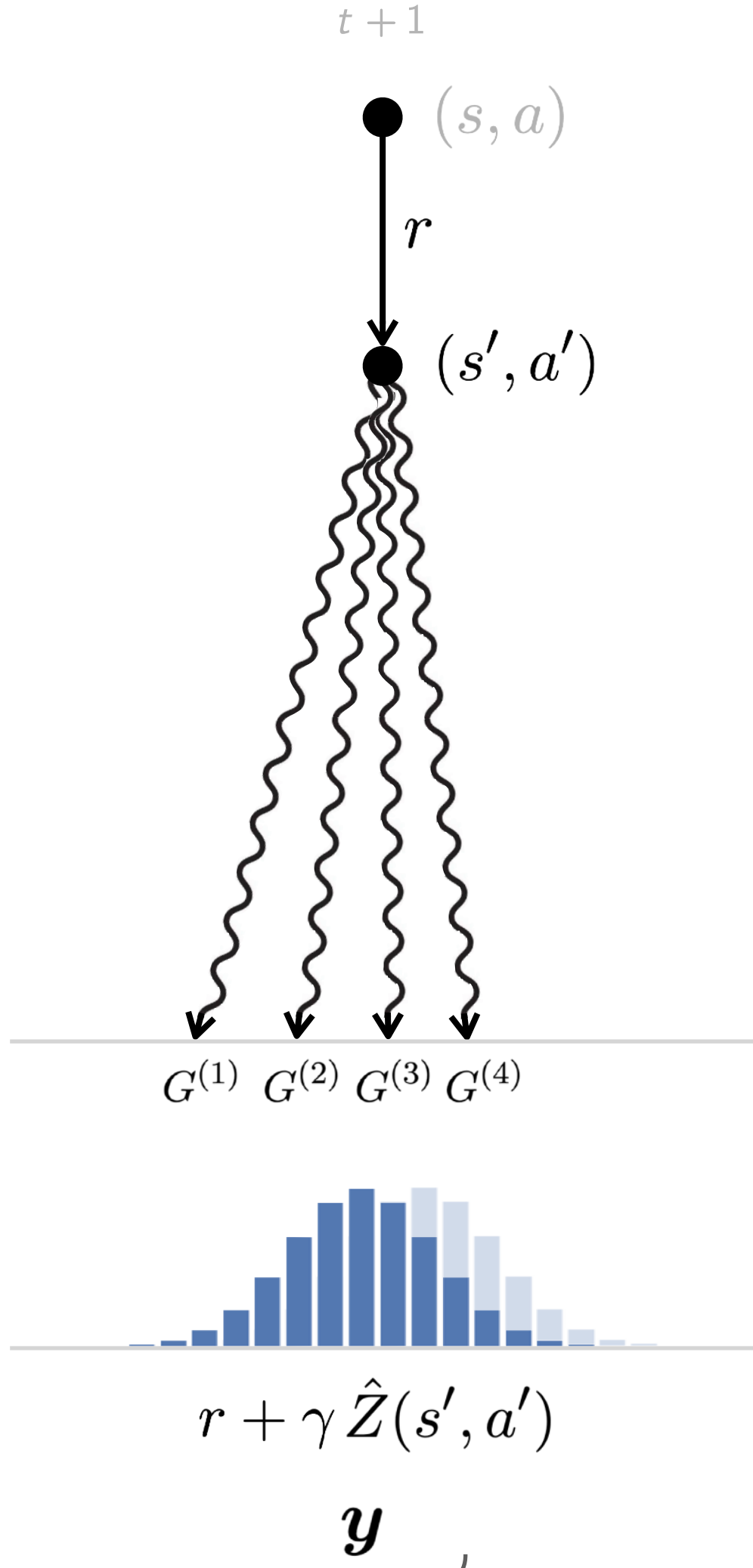
vs.

$$\mathbb{E} [H(y, \hat{y})]$$

Multiple rollouts just to show that the returns can end up being one of many values, and they follow some distribution.



Value distribution:



$$\begin{aligned} a' &= \arg \max_{a'} \bar{q}(s', a') \\ &= \arg \max_{a'} \mathbb{E} [\bar{Z}(s', a')] \end{aligned}$$

vs.

$$\mathbb{E} \left[-\mathbf{y}^\top \log \hat{\mathbf{y}} \right]$$

